

DESAFIO TÉCNICO: PROJETO ECOCYCLE

DESENVOLVIMENTO FULL-STACK (NODE.JS & WEB) PARA GESTÃO DE COLETA SELETIVA

Público-alvo: Alunos de Curso Técnico **Tecnologias:** Node.js, Express, HTML/CSS/JS

Tipo: Projeto de Revisão Prática

1. O Briefing / Contexto do Projeto

Você acabou de ser contratado como **Desenvolvedor(a) Full-Stack Júnior** pela **GreenTech Solutions**, uma startup focada em criar soluções tecnológicas para o desenvolvimento sustentável urbano.

A prefeitura de sua cidade acaba de fechar uma parceria com a GreenTech para resolver um grande problema de logística urbana: o descarte incorreto de resíduos recicláveis (eletrônicos, óleo de cozinha, plástico, papel, vidro e metal) e a falta de comunicação rápida entre os cidadãos e as cooperativas de reciclagem locais.

Sua primeira missão na equipe é desenvolver o MVP (Produto Mínimo Viável) da plataforma **EcoCycle**. O EcoCycle será um sistema web que permitirá a moradores locais cadastrar pontos de descarte temporários (ou pedir coleta de grandes volumes) e a cooperativas parceiras visualizar e gerenciar a retirada desses materiais.

Seu Papel no Projeto:

Como desenvolvedor responsável por esta entrega, você deverá planejar e executar desde o desenho arquitetural do banco de dados e do design de telas, até a codificação e documentação da API em Node.js e sua integração com a interface do usuário.

2. Etapas de Execução (O Fluxo de Trabalho Real)

Para garantir a qualidade que a GreenTech Solutions exige em seus projetos, você deverá seguir rigorosamente o fluxo de desenvolvimento oficial da empresa antes de sair programando:

- 1 Modelagem de Dados (Banco de Dados):** Desenhar o Modelo de Entidade e Relacionamento (MER) e, em seguida, gerar o Diagrama de Entidade-Relacionamento (DER) para estruturar a persistência dos dados (Usuários, Pontos de Descarte e Categorias de Resíduos).
- 2 Prototipagem (Figma):** Criar o Wireframe/Protótipo de alta ou média fidelidade da aplicação web no Figma para planejar a experiência do usuário (UX/UI) antes de estruturar o código front-end.
- 3 Desenvolvimento & Documentação da API:** Desenvolver o back-end usando Node.js + Express. O seu sistema deve obrigatoriamente estar documentado utilizando a especificação OpenAPI através do Swagger.

- 4 Integração Front-end:** Criar as telas com HTML5, CSS3 e JavaScript (ES6+), consumindo a API construída via requisições assíncronas (fetch ou axios).

3. Requisitos Funcionais (RF)

Os Requisitos Funcionais descrevem as ações e recursos que o sistema deve fornecer diretamente ao usuário final:

Código	Nome do Requisito	Descrição detalhada
RF-001	Cadastro de Usuários	O morador ou cooperativa deve conseguir se cadastrar fornecendo nome, e-mail, senha e tipo de perfil (Doador ou Coletor/Cooperativa).
RF-002	Autenticação	O usuário cadastrado deve poder realizar o login no sistema utilizando e-mail e senha.
RF-003	Cadastro de Pontos de Coleta	Um usuário "Doador" deve poder cadastrar um ponto de coleta contendo título, descrição do material, categoria do resíduo, endereço completo e quantidade estimada.
RF-004	Listagem de Pontos de Coleta	Qualquer usuário deve conseguir visualizar os pontos de coleta cadastrados que estão disponíveis para retirada.
RF-005	Atualização do Status da Coleta	Uma cooperativa/coletor cadastrado deve poder alterar o status de um ponto de coleta de "Pendente" para "Em Andamento" ou "Finalizado".

4. Requisitos Não Funcionais (RNF)

Os Requisitos Não Funcionais especificam os critérios técnicos, restrições e padrões de qualidade que o sistema deve cumprir:

Código	Atributo / Tecnologia	Critério de Aceitação Técnico
RNF-001	Ambiente Back-end	A API do servidor de aplicação deve ser desenvolvida utilizando a plataforma Node.js com o framework Express.
RNF-002	Persistência de Dados	Os dados devem ser salvos de forma relacional (ex: SQLite ou PostgreSQL) utilizando um ORM (Prisma, Sequelize) ou comandos nativos de SQL.
RNF-003	Arquitetura de API	A API deve seguir o padrão arquitetural REST, retornando dados e mensagens formatadas no padrão JSON.
RNF-004	Documentação Viva	Todas as rotas da API criadas devem estar documentadas e testáveis através da rota padrão /api-docs do Swagger.
RNF-005	Tecnologias Front-end	A interface web deve ser construída puramente utilizando HTML5 semântico, CSS3 (responsivo) e JavaScript (Vanilla) estruturado.



Dica do Tech Lead: Comece estruturando os endpoints básicos de cadastro de usuário e login. Em seguida, utilize a rota de login para gerar a lógica necessária para proteger os cadastros dos pontos de coleta!

5. Entregáveis Esperados do Desafio

Para validação e encerramento do projeto de MVP junto à prefeitura, você precisará organizar e submeter os seguintes artefatos técnicos no repositório Git de entrega:

1. **Repositório Git estruturado:** Com uma pasta /backend contendo os arquivos da API Node.js e uma pasta /frontend contendo os arquivos estáticos da web.
2. **Modelagem de Dados (MER e DER):** Arquivo de imagem ou link do software utilizado (ex: dbdiagram.io ou brModelo) descrevendo a estrutura de dados das tabelas.
3. **Link do Wireframe do Figma:** Adicionado ao arquivo README.md do repositório para consulta da identidade visual e layout propostos.
4. **Documentação Swagger:** Acessível rodando o projeto back-end localmente e estruturada por meio do pacote swagger-ui-express.
5. **Arquivo README.md Completo:** Instruções detalhadas de como instalar as dependências do back-end, criar/migrar o banco de dados e rodar os servidores localmente.